

Module 05: Action — Implementing Policy Selection and EFE

Course: Active Inference 101 | **Unit:** Implementation | **Audience:** First-semester undergraduates

Learning Objectives

1. Implement **multi-step policy evaluation** using Expected Free Energy.
2. Code the **pragmatic and epistemic value** components separately for analysis.
3. Implement **active inference with action** — closing the perception-action loop.

Key Concepts

1. Multi-Step Policy Generation

```
import numpy as np
from itertools import product

def generate_policies(num_actions, policy_length):
    """Generate all possible policies of given length."""
    return np.array(list(product(range(num_actions), repeat=policy_length)))

# Example: 3 actions, 2 steps = 9 policies
policies = generate_policies(3, 2)
print(f"Number of policies: {len(policies)}")
print(f"Example: {policies[0]}")
```

2. Full EFE with Decomposition

```

def compute_efe_decomposed(A, B, C, qs, policy):
    """
    Compute Expected Free Energy with pragmatic/epistemic decomposition.

    Returns:
        G: total EFE
        pragmatic: pragmatic value component
        epistemic: epistemic value component
    """
    pragmatic_total = 0
    epistemic_total = 0
    qs_pred = qs.copy()

    for t, action in enumerate(policy):
        # Predict future state
        qs_pred = B[:, :, action] @ qs_pred

        # Predict future observation
        qo_pred = A @ qs_pred

        # Pragmatic:  $-E_q[\log P(o)]$  – do predicted observations match pre
        prag = -(qo_pred * C).sum()
        pragmatic_total += prag

        # Epistemic: mutual information  $I(s;o)$  under predicted beliefs
        epist = 0
        for s in range(len(qs_pred)):
            if qs_pred[s] > 1e-16:
                po_s = A[:, s]
                epist += qs_pred[s] * (
                    po_s * (np.log(po_s + 1e-16) - np.log(qo_pred + 1e-16)

```

```

        ).sum()
        epistemic_total += epist

    G = pragmatic_total - epistemic_total
    return G, pragmatic_total, epistemic_total

def select_action(A, B, C, qs, policies, gamma=1.0):
    """Select action from evaluated policies."""
    G = np.zeros(len(policies))
    prag = np.zeros(len(policies))
    epist = np.zeros(len(policies))

    for i, policy in enumerate(policies):
        G[i], prag[i], epist[i] = compute_efc_decomposed(A, B, C, qs, pol

    # Softmax policy selection
    pi = np.exp(-gamma * G)
    pi /= pi.sum()

    # Sample policy
    chosen = np.random.choice(len(policies), p=pi)
    return int(policies[chosen][0]), G, prag, epist, pi

```

3. Active Inference Loop with Action

```

def active_inference_loop(A, B, C, D, env, num_steps=10, gamma=1.0, policies):
    """Full active inference loop with multi-step policies."""
    policies = generate_policies(B.shape[2], policy_len)
    qs = D.copy()

    log = {'beliefs': [], 'actions': [], 'obs': [], 'G': [], 'prag': [], 'epist': []}

    obs = np.random.choice(A.shape[0], p=A[:, env.state])

    for t in range(num_steps):
        # Infer states
        qs = A[obs, :] * qs
        qs /= qs.sum()

        # Select action
        action, G, prag, epist, pi = select_action(A, B, C, qs, policies, gamma)

        # Log
        log['beliefs'].append(qs.copy())
        log['actions'].append(action)
        log['obs'].append(obs)
        log['G'].append(G)
        log['prag'].append(prag)
        log['epist'].append(epist)

        # Act
        obs = env.step(action)

        # Transition beliefs
        qs = B[:, :, action] @ qs

```

```
print(f"t={t}: obs={obs}, act={action}, beliefs={qs.round(2)}")

return log
```

Summary

Multi-step policy evaluation generates all action sequences, computes EFE for each (decomposed into pragmatic and epistemic terms), and selects via softmax. The full active inference loop integrates perception, policy evaluation, and action into one continuous cycle.

Further Reading

- Sajid, N. et al. (2021). Active inference: Demystified and compared. *Neural Computation*, 33(3), 674-712.